ServiFinance: A Multi-Tenant SaaS Service Management System with Embedded Micro-Lending

In Partial fulfillment of the requirements
In IT13/L Professional Track for IT 4

Presented By:

Villacino, Christopher Llander L.

Presented To:

Lloyd Ryan Largo
IT13/L Adviser

April 2026

Part 1 – Project Title

ServiFinance Desktop Micro-Lending System

Part 2 - Project Analysis

1. Are the requirements of your system already clear or still changing? Why?

The core requirements are already clear because the project direction is well defined in the codebase and system documents: the desktop MLS must handle invoice-to-loan conversion, amortization, payment posting, ledger review, customer financial records, and audit visibility. However, some requirements are still changing at the refinement level. The current repo shows that major MLS workflows already exist, but reporting depth, premium entitlement visibility, finance hardening, and future multi-tenant employee access are still being improved. This means the system has a stable foundation, but some functional details and polish requirements are still evolving.

2. Does your project involve risk (hardware, security, complex process, new technology)? Explain.

Yes, the project involves several risks. It uses a hybrid setup where a `.NET MAUI` desktop application works with a shared React frontend and an ASP.NET Core backend, which adds integration complexity across channels. It also handles tenant authentication, desktop-only MLS access, financial transactions, amortization schedules, ledger balances, and payment posting, so security and data accuracy are critical. A mistake in tenant isolation, money calculations, or ledger updates could cause serious system errors. There is also platform and deployment risk because the desktop host must bootstrap and communicate with the backend reliably.

3. Does your system need frequent client/user feedback? Why?

Yes, frequent client or user feedback is needed. The MLS desktop workflow is used for finance-related actions, so the layout, process steps, reports, and validation messages must match how actual users expect to work. The repo itself shows ongoing refinement of reports, access states, and interface behavior, which means user feedback is important for deciding what should be simplified, clarified, or added next. Feedback is also needed because this system combines operational service work from SMS and finance work from MLS, so both workflow accuracy and usability matter.

4. Is your project individual or group based?

This project is individual based. The project documents describe the roadmap as a solo developer effort across a term, and the current implementation structure also reflects one coordinated codebase rather than separate team-owned subsystems. I also do the handling planning, coding, testing, and documentation, the process must support manageable iteration and clear priorities.

5. Is your deadline short, moderate, or flexible?

The deadline is moderate. The project is large enough to require multiple phases, but it is also intended to be completed within an academic term rather than under a very short crash deadline. The repo documents already break the work into staged implementation slices, which shows a schedule that allows iteration, testing, and adjustments. Because of that, the timeline is not fully flexible, but it is also not so short that only a rigid one-pass process would work.

Part 3 - Best Development Model

A. What model did you choose?

Agile / Scrum

B. Why is it the best for your project?

1. Agile fits the project type because ServiFinance is an ongoing business system with both web and desktop channels, so the work benefits from delivering usable modules step by step instead of waiting for one final release.

2. Agile fits the complexity because the system combines tenancy, authentication, invoicing, loans, amortization, collections, ledger tracking, and audit review, which are easier to build and verify in small increments.

3. Agile is strong for revisions because it shows a changing needs such as deeper reports, clearer entitlement handling, and finance hardening, and these are easier to absorb through short iterations than through a fixed one-time plan.

4. Agile works well for team size because this project is being developed individually, and a solo developer can still use sprint-style planning, backlog prioritization, and incremental delivery without the overhead of a heavy formal process.

5. Agile supports testing, timeline control, and feedback because each sprint can end with working features such as login, loan conversion, payment posting, or reporting, then those outputs can be checked, demonstrated, and improved before the next sprint starts.

C. Why are at least 2 other models less suitable?

Waterfall is less suitable because this project still has evolving details, especially in reporting, UI behavior, and finance hardening. A strict linear model would make later revisions more expensive and slower.

V-Model is also less suitable because it works best when requirements are fixed very early and each development phase maps neatly to a matching test phase. In this project, the

system is still being refined while modules are already being implemented, so a more iterative model is a better fit.

Part 4 - Development Plan Using Your Chosen Model

Agile / Scrum Plan

1. Sprint 1: Finalize requirements, review use cases, confirm desktop MLS scope, and prepare the shared data model and tenancy rules.

2. Sprint 2: Build the shared financial foundation, including invoices, micro-loans, amortization schedules, ledger transactions, and authentication support.

3. Sprint 3: Implement the desktop MLS access layer, including the `.NET MAUI` host, tenant desktop login, protected MLS routing, and backend connection flow.

4. Sprint 4: Develop core MLS workflows such as invoice-to-loan conversion, standalone loan creation, amortization generation, and customer financial records.

5. Sprint 5: Add payment posting, collections handling, ledger review, and audit review, then validate that balances and transaction trails are correct.

6. Sprint 6: Improve the reports module, blocked-state handling for non-premium access, validation rules, and finance safety checks.

7. Sprint 7: Perform full testing, bug fixing, interface cleanup, screenshot preparation, documentation updates, and final presentation readiness.

Part 5 - Reflection

Using a development model is better than coding immediately without planning because it gives the project a clear direction before major work begins. For a system like this (ServiFinance), planning helps organize complex features such as tenant security, loan processing, amortization, and ledger tracking in the correct order. It also reduces the chance of building the wrong feature first or creating logic that will need major rework later. A development model improves testing because every stage or sprint can be reviewed before the next one starts. It also makes time management easier because the project can be divided into realistic phases instead of becoming one large uncontrolled task. Most importantly, planning helps ensure that the final system is not only functional, but also secure, accurate, and aligned with user needs.